from *expressions*. Expressions can be composed of *sub-expressions*, and both are composed from *operations*.

The following list summarizes this hierarchy:

- Header statements
- Declaration statements
- General statements
- Expressions
- Sub-expressions
- Operations

Header statements are found at the beginning of every PHP file, immediately after the opening tag. These are limited to `declare()` statements (which control execution directives for a code block), `namespace` declarations, and `use` statements (the latter two are used to partition code, which is covered in detail in Chapter 3, "Functions").

General statements are all other possible statements that are not specific to PHP file header sections. A statement can be considered as a "line" of code and separated from other statements by the semicolon `;` character (it is the semicolon that delimits statements; you are free to use as much spacing, indentation, and new lines as you wish). Function and class declarations are also considered to be (special) statements in PHP.

Expressions (and sub-expressions) are sequences of operations that are *evaluated*: this means they can be substituted for values in the statement. For example, the expression `2 * 2;` evaluates to `4`, `"a string" . " and another string";` evaluates to `"a string and another string";`.

Operations are composed of *operators* and *operands*. Operators determine the type of operation; there are mathematical operators (such as `+`, `-`, `/`), logical operators (such as AND `&&`, OR `||`), and many others. Perhaps the most important is the assignment operator `=` that allows us to store and retrieve values. Operands are placed on either side of the operator; for example, `1 + 2` is a mathematical addition operation with operands `1` and `2`. Some operations only require a single operand, such as auto-increment: `1++;` has a single operand `1` and a single operator `++`. We will cover all of the operators in their relevant chapters.

We can further abstract operations in this way: operands can be *symbols* (also known as *labels*; these are the names of *variables*, *constants*, and *functions*) or *literal* values.